Shoply

Full-Stack E-Commerce Web Application

Technology Stack: React.js · Java Spring Boot · MySQL

Author: Soham Samanta · GitHub: soham29640

Repository: github.com/soham29640/Shoply

Document Version: 1.1 | April 2025

Table of Contents

1. Problem Statement
2. Solution & Features
3. Screenshots
4. Tech Stack
5. Unique Points
6. Future Improvements

---

1. Problem Statement

Online shopping has become an essential part of modern commerce, yet building a production-quality e-commerce platform from scratch remains a complex engineering challenge. Existing open-source storefronts are often either too heavyweight (requiring extensive infrastructure) or too minimal (missing authentication, cart persistence, and order management). There is a clear need for a clean, well-structured reference implementation that demonstrates how a real-world, full-stack shopping application is designed, built, and deployed.

Specific challenges addressed:

- **Secure user authentication:** Most tutorials skip JWT-based stateless authentication or BCrypt password hashing. A production-ready implementation is needed.
- **Cart & state management:** Client-side cart state must survive page refreshes and integrate seamlessly with a server-side order workflow.
- **Product discoverability:** Users need instant full-text search, category filtering, and price filtering without reloading the page.
- **Order lifecycle:** Placing an order must atomically persist all line items, shipping details, and payment metadata in a relational database.
- **Tech stack cohesion:** Connecting a React SPA to a Spring Boot REST API with correct CORS policy, JWT handshake, and Axios interceptors requires careful integration.

2. Solution & Features

Shoply is a feature-complete full-stack e-commerce web application that mirrors the core shopping experience of Amazon.com. It is built with React.js on the frontend and Java Spring Boot on the backend, connected via a RESTful JSON API secured with JSON Web Tokens (JWT). The application is designed to be modular, easily extensible, and deployable with a single command per tier.

Core Features

- **User Registration & Login:** Secure sign-up and sign-in with BCrypt-hashed passwords and JWT token issuance. Tokens are stored in localStorage and auto-attached to every API request via an Axios interceptor.
- **Product Catalogue:** 12 seeded products across multiple categories (Electronics, Clothing, Books, etc.) with name, description, price, original price (for discount display), brand, rating, stock level, and Prime eligibility flag.
- **Full-Text Search:** Real-time search across product names and descriptions using Spring Data JPA's LIKE query. Results update instantly as the user navigates to /search.
- **Category & Price Filtering:** Users can filter the homepage product grid by category or set a maximum price. Filters are applied server-side for accuracy.
- **Shopping Cart:** Persistent cart management via React Context API with localStorage fallback. Cart item count badge in the Navbar updates live. Backend /api/cart endpoints support server-side cart for authenticated users.
- **Checkout & Order Placement:** A multi-field checkout form collects shipping name, phone, address, and payment method. On submission, a POST /api/orders call persists the order atomically; the cart is then cleared.
- **Order History:** Authenticated users can view all past orders at /orders, with each order showing status, total, items, and shipping details.
- **Admin Product Management:** ADMIN-role users can create, update, and delete products via protected REST endpoints (POST/PUT/DELETE /api/products).
- **Responsive UI:** All pages use CSS Flexbox/Grid and are designed to reflow gracefully on tablet and mobile viewports.

3. Screenshots

The following screenshots illustrate the main pages and user flows of the application. All UI is rendered by the React.js frontend communicating with the Spring Boot REST API.

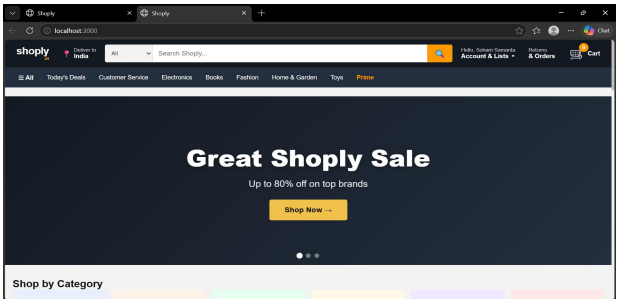

Figure 1: Home Page — Product Grid with Category Filter

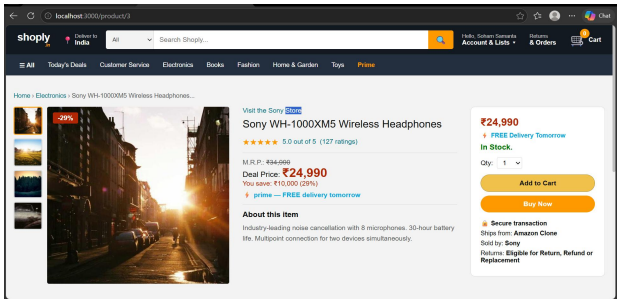

Figure 2: Product Detail Page

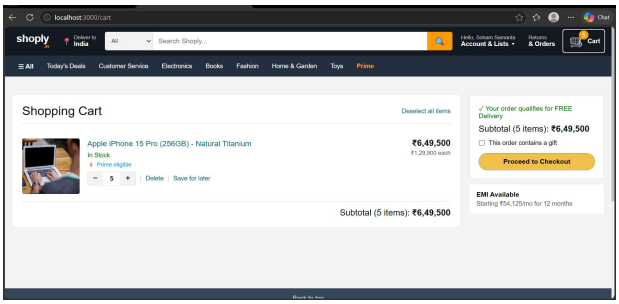

Figure 3: Shopping Cart Page

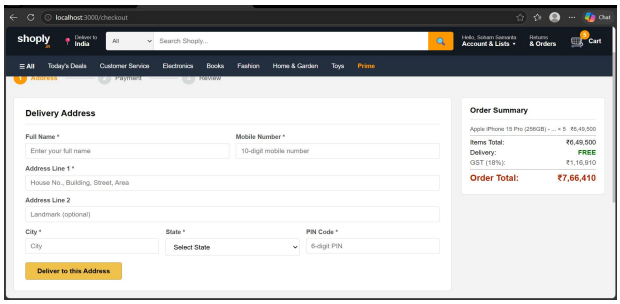

Figure 4: Checkout Page

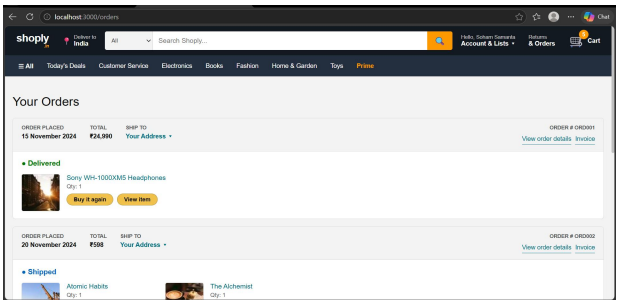

Figure 5: Order History Page

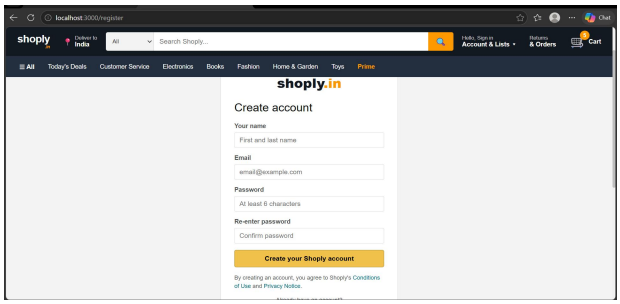

Figure 6: Login / Register Pages

4. Tech Stack

The application follows a standard three-tier architecture: a React single-page application (SPA) as the presentation layer, a Spring Boot REST API as the business logic layer, and MySQL as the data layer.

Layer	Technology	Purpose
Frontend	React.js 18	Component-based UI; Context API for global state
Frontend	React Router v6	Client-side routing (BrowserRouter, Routes, Link)
Frontend	Axios	HTTP client; request interceptor auto-attaches JWT
Frontend	CSS3 / Flexbox	Responsive layout and styling
Backend	Java 17	Core language; records, text blocks, switch expressions
Backend	Spring Boot 3.2	Auto-configuration, embedded Tomcat, bean lifecycle
Backend	Spring Security 6	Filter chain, CORS, route-level access control
Backend	Spring Data JPA	Repository abstraction over Hibernate ORM
Auth	JWT (jjwt 0.11.5)	Stateless token-based authentication
Auth	BCrypt	Password hashing with adaptive cost factor
Database	MySQL 8	Relational RDBMS for persistent data
Build	Maven 3.8	Backend dependency management and build lifecycle
Build	npm 9 / Node 18	Frontend dependency management and dev server
Dev Tools	Lombok	Reduces boilerplate (@Data, @Builder, @Slf4j)
Dev Tools	Spring DevTools	Auto-restart on class change during development

5. Unique Points

- **End-to-End JWT Security:** Fully stateless API — JwtAuthFilter validates every protected request independently, enabling horizontal scalability without sticky sessions.
- **MySQL Persistence:** Full relational persistence with Hibernate ddl-auto=update for safe schema evolution without data loss.
- **Context API + localStorage Hybrid Cart:** Instant UI feedback on cart interactions; state survives page refreshes without extra API calls.
- **Layered Architecture:** Strict Controller → Service → Repository separation; no business logic in controllers, no DB queries in services.
- **Seed Data via data.sql:** 12 realistic products loaded at startup — new contributors see a meaningful catalogue with zero manual setup.
- **Role-Based Admin API:** ADMIN-only product CUD endpoints enforced at Spring Security layer; unauthorized access returns HTTP 403.
- **Axios Request Interceptor:** Centrally maintained interceptor in utils/api.js attaches JWT to every request; feature components stay clean and reusable.
- **Prime Eligibility Flag:** is_prime boolean on products enables future Prime-filtered listings via the existing GET /api/products/prime endpoint.

6. Future Improvements

- **Payment Gateway Integration:** Integrate Stripe or Razorpay for real credit/debit card and UPI payments, replacing current placeholders.

• **Product Reviews & Ratings UI:** Complete the frontend review submission form and rating histogram — the backend Review entity already exists.

• **Real-Time Order Tracking:** Add WebSocket (STOMP over SockJS) support to push order-status updates (Placed → Shipped → Delivered) to the browser.

• **Admin Dashboard:** Dedicated /admin panel with charts (orders per day, revenue, top products), CRUD forms, and user management for ADMIN-role accounts.

• **Image Upload via Cloud Storage:** Replace static image_url strings with a file-upload workflow backed by AWS S3 or Cloudinary.

• **Wishlist / Saved Items:** Allow authenticated users to save products server-side with one-click 'Move to Cart' action.

• **Email Notifications:** Transactional emails (order confirmation, shipping update) via Spring Boot Mail with Thymeleaf HTML templates.

• **Containerisation & CI/CD:** Dockerfile + docker-compose.yml for one-command local startup; GitHub Actions workflow for automated build, test, and Docker Hub push on merge.

• **Pagination & Infinite Scroll:** Server-side pagination (Spring Data Pageable) paired with infinite-scroll component for large product catalogues.

• **Accessibility & Internationalisation:** WCAG 2.1 AA audit, ARIA labels, and i18next for multi-language support (English, Hindi, Spanish).

This document was prepared as part of the Shoply project submission. All screenshots are from the live application.